Région Hauts-de-France
Cofinancé par l'Union européenne
realconseil DIAGNOSTIC ACCOMPAGNEMENT FORMATION

Exercices POO JavaScript (Le Seigneur de la POO)

Objectifs

Développer la compréhension des classes, de l'héritage, du polymorphisme et de la composition à travers des personnages et peuples de fantasy.

Exercice 1 – Classe simple : Personnage

Créer une classe `Personnage` avec :

- un constructeur prenant `nom` et `race`
- une méthode `sePresenter()` affichant : « Je suis NOM, de la race RACE. »

Créer trois instances :

- un elfe nommé Elandril
- un nain nommé Bofur
- un orque nommé Krug

Appeler `sePresenter()` pour les trois.

Exercice 2 – Ajouter un comportement générique

Ajouter à `Personnage` une méthode `agir()` qui affiche : « NOM se tient prêt à agir. »

Créer un hobbit nommé *Pip* et appeler `agir()`.

Exercice 3 – Héritage : Guerrier, Mage, Voleur

Créer trois classes spécialisées :

- `Guerrier` (hérite de `Personnage`)
- `Mage` (hérite de `Personnage`)
- `Voleur` (hérite de `Personnage`)

Le constructeur doit définir automatiquement la race *par défaut*, par exemple :

- Guerrier → `race = "Humain"`
- Mage → `race = "Elfe"`
- Voleur → `race = "Hobbit"`

Créer :

- un guerrier nommé Arondir
- un mage nommé Faelivrin
- un voleur nommé Ludo

Tester `sePresenter()`.

Exercice 4 – Polymorphisme

Redéfinir la méthode `agir()` dans :

- `Guerrier` → « NOM brandit son arme. »
- `Mage` → « NOM murmure une incantation ancienne. »
- `Voleur` → « NOM disparaît dans l'ombre. »

Créer un tableau de personnages : un guerrier, un mage, un voleur, et un orque classique (classe de base).

Boucler dessus et appeler `agir()` pour observer le polymorphisme.

Exercice 5 – Encapsulation

Créer une classe `Combattant` héritant de `Personnage` avec :

- un attribut privé `#pointsDeVie` initialisé à 100
- une méthode `subirDegats(valeur)` qui retire des points de vie
- une méthode `soigner(valeur)` qui en ajoute (sans dépasser 100)
- un getter `pv` pour obtenir les points actuels

Créer un Uruk-hai nommé *Gorash*, lui infliger 35 points de dégâts, afficher ses PV, puis le soigner de 20.

Exercice 6 – Composition

Créer une classe `Monture` avec :

- `type` (cheval, warg, poney, grand cerf elfe…)
- `nom`
- une méthode `decrire()`

Créer une classe `Cavalier` héritant de `Personnage` avec :

- un attribut `monture`
- une méthode `seDeplacer()` qui affiche la présentation du personnage et celle de la monture.

Créer :

- un Rohirrim nommé *Eofred* monté sur un cheval nommé *Vent-du-Nord* Appeler `seDeplacer()`.

Exercice 7 – Mini-projet : Armée de la Terre du Milieu

Créer la hiérarchie :

- `Personnage`
- `Humain`, `Elfe`, `Nain`, `Orque`, `UrukHai` (héritent de Personnage)
- Facultatif : `Ent`, `Hobbit`

Créer 8 instances mélangeant les races. Mettre toutes les instances dans un tableau.

Écrire trois fonctions :

1. afficher uniquement les elfes
2. afficher les personnages dont le nom commence par « A »
3. appeler `sePresenter()` et `agir()` sur tous les personnages